

Java Engineer Online Assessment - Solutions

PART I: Fundamentals

Question 1: Java Collections & Thread Safety

1. Internal Data Structure of HashMap in Java 8+ and Hash Collision

In Java 8+, a HashMap is implemented using a **Node array (bucket table)** combined with **Singly Linked Lists** and **Red-Black Trees**.

- **Hash Collision Handling:** When two different keys produce the same hash bucket index, they are chained together.
- **Treeification:** To prevent Performance Degradation (reducing lookup from $O(n)$ to $O(\log n)$), if the number of elements in a single bucket exceeds the treeify threshold (`TREEIFY_THRESHOLD = 8`) and the overall capacity of the table is at least `MIN_TREEIFY_CAPACITY = 64`, the linked list is converted into a balanced **Red-Black Tree**. If the table capacity is less than 64 , the map simply resizes (doubles in size) instead of treeifying.

2. Concurrency Issues with `HashMap.put()`

HashMap is not thread-safe. When multiple threads call `put()` concurrently on the same instance, several critical failures can occur:

- **Scenario (Data Loss/Race Condition):** Suppose Thread A and Thread B concurrently call `put()` with different keys that resolve to the same empty bucket index i . Both threads read `table[i]` as `null` at the same time. Thread A writes its node to `table[i]`, and immediately after, Thread B writes its node to `table[i]`. Thread A's entry is silently overwritten and permanently lost.
- **Size Inconsistency:** The internal size counter is incremented non-atomically (`size++`). Concurrent writes will lead to lost updates on the count, resulting in an incorrect size value.
- **Infinite Loops (Java 7 and earlier):** Though resolved in Java 8 due to tail-insertion, concurrent resizing in earlier versions could cause circular references in the linked lists, causing subsequent `get()` operations to hang in an infinite loop, driving CPU usage to 100% .

3. ConcurrentHashMap vs. Collections.synchronizedMap()

- **Collections.synchronizedMap()**: Wraps the standard HashMap and synchronizes every operation (reads and writes) on a single mutual exclusion lock (the map instance itself). This forces all threads to wait in a single queue, creating a massive serialization bottleneck.
- **ConcurrentHashMap (Java 8+)**: Achieves thread safety with high concurrency using a fine-grained locking strategy:
 1. **Node-Level Locking (Bucket Locking)**: Instead of locking the entire map, it uses the synchronized keyword on the head node of each individual bucket.
 2. **Lock-Free CAS (Compare-And-Swap)**: When inserting into an empty bucket, it uses low-level atomic CAS operations (via Unsafe) without acquiring any locks.
 3. **Volatile Reads**: Read operations (get()) do not lock at all because the node values and the next pointers are declared volatile, ensuring immediate cross-thread visibility of modifications.

Question 2: OOP Design Principles

1. Open-Closed Principle (OCP)

The **Open-Closed Principle** states that software entities (classes, modules, functions) should be open for extension but closed for modification. To follow OCP in our notification system, we define a common interface (NotificationChannel). When a new channel (e.g., Push, WhatsApp) is introduced, we only need to write a new class implementing this interface. The core dispatching code remains completely untouched.

2. Dependency Inversion Principle (DIP)

The **Dependency Inversion Principle** states that high-level modules should not depend on low-level modules; both must depend on abstractions. Instead of OrderService directly instantiating and calling EmailNotification or SmsNotification classes, it depends on the NotificationService abstraction. The actual implementations are injected into OrderService at runtime using Dependency Injection (DI).

3. Java Code Skeleton

```
// Abstraction for a single delivery channel
public interface NotificationChannel {
    void send(String recipient, String message);
    String getChannelType();
}

// Low-level implementations
public class EmailChannel implements NotificationChannel {
```

```

    @Override
    public void send(String recipient, String message) {
        // Email-specific implementation
    }

    @Override
    public String getChannelType() { return "EMAIL"; }
}

public class SmsChannel implements NotificationChannel {
    @Override
    public void send(String recipient, String message) {
        // SMS-specific implementation
    }

    @Override
    public String getChannelType() { return "SMS"; }
}

// High-level service orchestrator
public interface NotificationService {
    void notify(String recipient, String message, String channelType);
}

public class NotificationServiceImpl implements NotificationService {
    private final Map<String, NotificationChannel> channels;

    // Channels are dynamically injected via Spring/DI framework
    public NotificationServiceImpl(List<NotificationChannel> channelList) {
        this.channels = channelList.stream()
            .collect(Collectors.toMap(NotificationChannel::getChannelType,
c -> c));
    }

    @Override
    public void notify(String recipient, String message, String
channelType) {
        NotificationChannel channel =
channels.get(channelType.toUpperCase());
        if (channel == null) {
            throw new IllegalArgumentException("Unsupported channel: " +
channelType);

```

```

    }
    channel.send(recipient, message);
}
}

// Business Logic Layer depending strictly on the abstraction
public class OrderService {
    private final NotificationService notificationService;

    public OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void completeOrder(String userId, String orderId) {
        // Business logic here...
        notificationService.notify(userId, "Your order " + orderId + " is
complete!", "EMAIL");
    }
}

```

Question 3: JVM Memory Model & Garbage Collection

1. JVM Runtime Memory Areas

- **Heap:** Shared across all threads. Stores all objects, arrays, and instance variables. It is the primary target for GC.
- **JVM Stack (Thread Stack):** Thread-private. Stores stack frames containing local variables, primitive data, object references, and method execution states.
- **Metaspace:** Shared across all threads. Allocated from native memory (since Java 8). Stores class metadata, runtime constant pools, and method definitions.
- **Program Counter (PC) Register:** Thread-private. Stores the memory address of the JVM instruction currently being executed by the thread.
- **Native Method Stack:** Thread-private. Stores state and local variables for native methods written in C/C++.

2. Generational Heap Model & Weak Generational Hypothesis

The heap is divided into the **Young Generation** (Eden, Survivor Space S0, and Survivor Space S1) and the **Old Generation**.

- **Weak Generational Hypothesis:** The design is built on two observations:
 1. Most allocated objects live for a very short time (they become unreachable soon after allocation).
 2. Objects that survive multiple garbage collection cycles tend to live for a long time.
- **Why Generational GC?** It allows the JVM to apply different GC algorithms to different areas. The Young Gen uses copy-based collection (fast, compacts memory, fits high infant mortality), while the Old Gen uses Mark-Sweep-Compact algorithms which avoid copying large, long-lived objects repeatedly.

3. Diagnosis and Analysis of the GC Log Pattern

- **Analysis:**
 1. Young GC is fast (15ms) and runs regularly, meaning the allocation rate in Eden is steady and healthy.
 2. Full GC is extremely frequent (every 30 seconds) and takes a long time (800ms).
 3. Critically, the Old Gen occupancy only drops from 85% to 78% after a Full GC. Only 7% of the Old Gen is actually reclaimed.
- **Root Cause:** There is a **Memory Leak** in the Old Generation, or the active long-lived object footprint of the application is too large for the allocated heap size. The JVM is thrashing, spending major CPU cycles executing consecutive Full GCs to free tiny amounts of space.
- **Investigation Steps:**
 1. Capture a heap dump during a peak load period using `jcmd <pid> GC.heap_dump /path/to/dump.hprof` or `jmap`.
 2. Analyze the heap dump using Eclipse Memory Analyzer (MAT) or JProfiler to find the path to GC roots for the objects dominating the 78% retained space.
 3. Look for growing collections (e.g., static lists, cache structures without TTL/eviction, unclosed connections, or uncleared `ThreadLocal` variables).
- **JVM Tuning Adjustments:**
 1. Increase the overall heap size (`-Xmx` and `-Xms`) if the high memory footprint is a legitimate business requirement.
 2. Switch to a modern GC algorithm like **G1GC** (`-XX:+UseG1GC`) or **ZGC** (`-XX:+UseZGC`) designed to handle large heaps with lower pause times.
 3. Adjust the G1 mixed collection initiation threshold if using G1 (`-XX:InitiatingHeapOccupancyPercent`).

Question 4: Concurrency & Thread Coordination

1. volatile vs. synchronized

- **volatile** is a lightweight synchronization mechanism. It guarantees **visibility** (any write is immediately written to main memory and visible to other threads) and **ordering** (prevents instruction reordering). It does *not* guarantee atomicity.
- **synchronized** is a heavyweight lock. It guarantees **visibility**, **ordering**, and **atomicity** (mutual exclusion block; only one thread can execute the locked block at a time).

Volatile Sufficient Scenario: A thread shutdown flag.

```
private volatile boolean running = true;
public void shutdown() { running = false; }
```

-

Volatile Insufficient Scenario: A shared counter increment.

```
private volatile int count = 0;
public void increment() { count++; } // Not atomic (read-modify-write)
```

-

2. Deadlock Scenario Code

```
public class DeadlockDemo {
    private static final Object resource1 = new Object();
    private static final Object resource2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (resource1) {
                try { Thread.sleep(50); } catch (InterruptedException
ignored) {}

                synchronized (resource2) {
                    System.out.println("Thread 1 acquired both resources");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (resource2) {
                try { Thread.sleep(50); } catch (InterruptedException
ignored) {}

                synchronized (resource1) {
                    System.out.println("Thread 2 acquired both resources");
                }
            }
        });
    }
}
```

```
});

t1.start();
t2.start();
}
}
```

3. Thread Coordination Utilities

- **Option A: `CountDownLatch(10)`**
 - *Mechanism:* The main thread blocks using `latch.await()`. Each of the 10 worker threads decrements the count using `latch.countDown()` upon completion.
 - *Trade-offs:* Very simple and lightweight. However, it is a one-use utility; the count cannot be reset or reused once it reaches zero.
- **Option B: `CompletableFuture.allOf(...)`**
 - *Mechanism:* Launch 10 tasks as `CompletableFuture` objects, and block/join on `CompletableFuture.allOf(futuresArray)`.
 - *Trade-offs:* Highly expressive and integrates perfectly with functional programming/reactive designs. It allows easy error propagation and chaining of secondary pipelines, but introduces slightly more design overhead than a basic latch.

Question 5: Database Index Design & Query Optimization

1. Optimal Composite Index Design

For the given query:

```
SELECT merchant_id, SUM(amount) as total_amount, COUNT(*) as tx_count
FROM transactions
WHERE status = 'SUCCESS'
  AND channel = 'ONLINE'
  AND created_at BETWEEN '2025-01-01' AND '2025-03-31'
GROUP BY merchant_id
ORDER BY total_amount DESC
LIMIT 50;
```

The optimal composite index is: (**status, channel, created_at, merchant_id, amount**)

- **Column Order Explanation:**

1. **status and channel (First):** Since these are matched with equality operators (=), they must appear first. This reduces the search space immediately.
2. **created_at (Third):** This is a range query (BETWEEN). Once a range column is used in a composite index, any subsequent column in the index cannot be used for multi-column index filtering.
3. **merchant_id (Fourth):** Placing this after created_at supports the GROUP BY merchant_id clause (covering index optimization), avoiding file sorts if the optimizer can leverage the index order.
4. **amount (Fifth):** Adding amount to the end of the index turns it into a **Covering Index**. This allows the engine to compute SUM(amount) entirely from the index tree without fetching the actual rows from disk.

2. Leftmost Prefix Rule Evaluation

Using the index (status, channel, created_at, merchant_id):

- WHERE status = 'SUCCESS': **Benefits fully**. It matches the first column of the index.
- WHERE channel = 'ONLINE': **Does not benefit**. The first column of the composite index (status) is missing; the database must perform a full table scan.
- WHERE status = 'SUCCESS' AND created_at > '2025-01-01': **Benefits partially**. It uses the index for the status filter, but because channel (the second column) is missing, it cannot perform index-range scans on created_at.
- WHERE channel = 'ONLINE' AND status = 'SUCCESS': **Benefits fully**. The MySQL query optimizer will reorder the conditions to match the physical index order (status, channel).

3. Covering Index Definition and Applicability

- **Definition:** A covering index is an index that contains all the columns required by the query (including columns in SELECT, WHERE, GROUP BY, and ORDER BY). When a covering index is present, the database engine reads data solely from the index page structures in memory, completely bypassing physical row lookup (table space access).
- **Applicability:** Yes, the proposed index (status, channel, created_at, merchant_id, amount) is a covering index for this query. Every single field utilized in the query is physically located inside the index.

PART II: Comprehensive Problem Solving

Question 6: System & Database Design

1. Single-Table Design Evaluation

Evaluating the proposed wallet_transactions table where current balance is computed on the fly:

1. **Severe Concurrency & Race Conditions:** To process a new transaction, the system must query the previous row's balance_after. Under concurrent requests, two threads could read the same "latest" transaction balance, calculate a new balance, and insert records. This leads to silent balance inconsistencies and overdrafts.
2. **Exponentially Slow Balance Queries ($O(N)$ lookup):** Finding the latest balance requires sorting all historical entries for a user: `ORDER BY created_at DESC LIMIT 1`. As the history table scales to millions of rows, the $O(N)$ balance inquiry requirement becomes impossible to satisfy.
3. **High Vulnerability to Corrupted Data:** Because there is no source of truth table for balance, if a single historical transaction row is accidentally deleted or corrupted, the current balance and all subsequent calculations for that user are permanently compromised.

2. Improved Database Schema (DDL)

We normalize the data by splitting the current balance state into a highly optimized wallets table and the transaction history into a separate wallet_transactions table.

```
-- User Wallet Table (Source of truth for balances)
CREATE TABLE wallets (
  user_id BIGINT PRIMARY KEY,
  balance DECIMAL(15,4) NOT NULL DEFAULT 0.0000,
  currency VARCHAR(3) NOT NULL DEFAULT 'USD',
  version INT NOT NULL DEFAULT 0, -- Used for optimistic locking if
needed
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- Wallet Ledger/Transactions Table (Insert-only history)
CREATE TABLE wallet_transactions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  transaction_no VARCHAR(64) UNIQUE NOT NULL, -- For idempotency
```

```

from_user_id BIGINT, -- NULL for top-up
to_user_id BIGINT,   -- NULL for withdrawal
type VARCHAR(20) NOT NULL, -- 'TOP_UP', 'TRANSFER', 'PAYMENT'
amount DECIMAL(15,4) NOT NULL,
description VARCHAR(255),
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
KEY idx_from_user_created (from_user_id, created_at),
KEY idx_to_user_created (to_user_id, created_at)
);

```

3. Balance Consistency Design (Transfer Scenario)

To process a transfer of \$100.00\$ from User A (user_id = 1) to User B (user_id = 2):

1. **Transaction Wrapping:** Wrap the entire operation in a database transaction (@Transactional(isolation = Isolation.READ_COMMITTED)).
2. **Deadlock Prevention:** Always lock rows in a consistent order (e.g., lock the smaller user_id first).
3. **No Overdraft / Concurrency Control:** Use Pessimistic Locking (SELECT FOR UPDATE) to prevent concurrent updates on the wallet balance.

```

-- Step 1: Lock both records in ascending order of user_id to prevent
deadlocks
SELECT balance FROM wallets WHERE user_id = 1 FOR UPDATE;
SELECT balance FROM wallets WHERE user_id = 2 FOR UPDATE;

-- Step 2: Validate User A's balance in application logic
-- If (balance_A >= 100.00) then proceed, else rollback and throw
InsufficientFundsException

-- Step 3: Deduct User A and Add to User B
UPDATE wallets SET balance = balance - 100.00 WHERE user_id = 1;
UPDATE wallets SET balance = balance + 100.00 WHERE user_id = 2;

-- Step 4: Write ledger record
INSERT INTO wallet_transactions (transaction_no, from_user_id, to_user_id,
type, amount, description)
VALUES ('TX_9981249', 1, 2, 'TRANSFER', 100.00, 'Transfer to User B');

```

```
-- Commit transaction
```

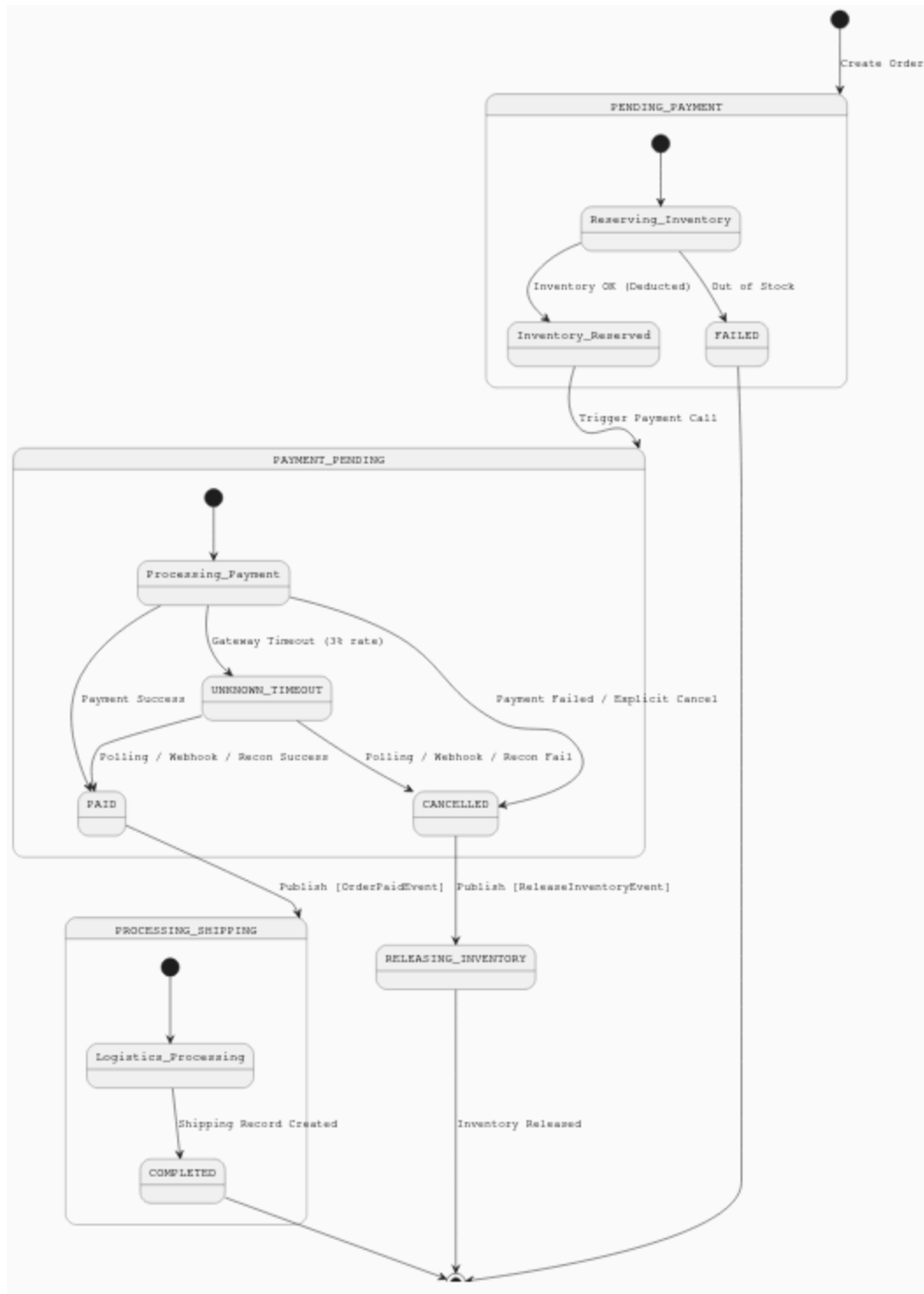
4. Query Performance Trade-offs

- **Real-time Balance Inquiry (< 10ms):** Satisfied by executing a simple Primary Key lookup on the optimized wallets table (SELECT balance FROM wallets WHERE user_id = ?). Since this is a small table with a primary key index, lookup is sub-millisecond.
- **Transaction History Pagination (< 200ms):** Satisfied using the composite indexes idx_from_user_created and idx_to_user_created. When a user requests history, we use keyset pagination (e.g., WHERE from_user_id = ? AND created_at < ? ORDER BY created_at DESC LIMIT 20) which ensures fast index scans even at millions of rows.
- **Trade-off Analysis:** Storing current balance redundantly in wallets is superior. Although it requires strict write synchronization with wallet_transactions, it achieves exceptional read throughput and eliminates the massive database CPU/IO bottlenecks of aggregate queries on historical tables.

Question 7: Distributed Data Consistency Design

1. Overall Workflow and State Transitions

Below is the order state transition workflow designed to handle eventual consistency, represented as a PlantUML state diagram:



2. Handling Payment Gateway Timeout

When the payment gateway times out (3% rate):

1. **Uncertain State:** Transition the Order Status to PAYMENT_PENDING (never assume failure or success).
2. **Active Polling (Query Task):** Execute an asynchronous scheduled worker that queries the payment gateway's order status endpoint using the unique local payment transaction ID.
3. **Gateway Callbacks (Webhooks):** Set up a highly available webhook endpoint to consume asynchronous notifications from the gateway.
4. **Daily Reconciliation:** Run a nightly batch reconciliation job comparing local state with the gateway's settlement file.
5. **Idempotency:** The query and callback processors must use a unique payment transaction ID to ensure that processing the same success notification multiple times is idempotent.

3. Handling Partial Failures (Logistics Down)

- **Eventual Consistency via MQ:** When the Payment Service receives confirmation of success, it publishes an OrderPaidEvent to a reliable Message Queue (e.g., RabbitMQ or Kafka) configured with persistent queues and manual acknowledgments.
- **Logistics Recovery:** The Logistics Service consumes this event. If the Logistics Service is offline, the message remains safely queued in the broker. Upon recovery (even if it takes 2 hours), the service will consume the queue and generate the shipping records.
- **Long-Term Outage Mitigation:** If the service is down for an extended period:
 - The queue buffer protects the database from overloading.
 - A dead-letter queue (DLQ) threshold is set to catch persistently failing records.
 - A user-facing status page shows: *"Your payment is confirmed. Shipment processing is currently delayed."*

4. Preventing Inventory Overselling (5000 orders/min)

- **Strategy:** Use **Redis-based Atomic Pre-Deduction** with a fallback to DB inventory updates.
- **Implementation Details:**
 1. Store product inventory counts inside Redis.

When an order is processed, execute a Lua Script to check and decrement the count atomically:

```
local key = KEYS[1]
local quantity = tonumber(ARGV[1])
local current_stock = tonumber(redis.call('get', key) or "0")
if current_stock >= quantity then
    return redis.call('decrby', key, quantity)
else
    return -1
end
```

end

- 2.
 3. If Redis returns a value ≥ 0 , the inventory reservation succeeds, and the order proceeds. Write the database decrement asynchronously via a message queue.
 4. If Redis returns -1 , fail immediately (protecting the primary database from load).
- **Trade-off:** High performance and sub-millisecond throughput. The trade-off is the complexity of synchronizing cache and database state, and the requirement of a robust cache recovery system if Redis crashes.

Question 8: System Performance Diagnosis & Optimization

1. Performance Diagnosis

1. **High Memory Heap Spike (OOM Risk):** The application is likely querying and mapping the entire monthly list of successful transaction records from MySQL directly into the JVM heap. This loads millions of domain entities into memory at once, causing severe GC pauses and OutOfMemoryErrors.
2. **High Database CPU (95%+):** Executing complex, un-indexed aggregations and scans over a $200 \times M$ row table causes severe disk I/O and CPU bottlenecks on the primary master database.
3. **No Thread and Service Isolation:** The batch process runs on the same application server and uses the same thread pool and database connection pool as the user-facing APIs. The resource starvation cascades down, degrading latency across the system.
4. **No Paginated/Checkpoint Processing:** Generating the report as a monolithic, single-run process means that if the job fails halfway through, all computational progress is lost, forcing a complete rerun.

2. Optimization Plan

- **Avoid Loading Everything to Memory:** Implement **Streaming/Cursor Queries** (e.g., MyBatis Cursor, or Spring JDBC RowCallbackHandler with a custom fetch size like 1000). Alternatively, use chunk-based processing via **Spring Batch**.
- **Reduce Database Load:** Direct all analytical query read traffic to a dedicated **Read Replica** database, keeping the primary write master database completely unaffected.
- **Parallelization:** Implement a master-worker partition pattern. Divide the list of merchants into multiple chunks and run calculations concurrently across a thread pool (using Spring Batch's Partitioning or custom ExecutorService).
- **Workload Isolation:** Deploy the reporting API/job on a **dedicated worker service instance** with its own CPU/Memory allocation and separate database connection pools.

3. Database Query Strategy (200M+ Rows)

- **Partitioning:** Partition the transactions table physically by month based on the created_at column. This restricts scans to the exact monthly partition rather than scanning 200M rows.

Batch Retrieval by Keyset: Retrieve transaction rows in chunks using indexed primary keys rather than offset paging:

```
SELECT id, merchant_id, amount, fee
FROM transactions
WHERE status = 'SUCCESS'
  AND created_at BETWEEN '2025-01-01' AND '2025-01-31'
  AND id > :last_processed_id
ORDER BY id ASC
LIMIT 10000;
```

-
- **Read Replica Utilization:** Yes, this is mandatory. An analytical job performing mass aggregations will block OLTP resources, cause locking, and consume execution pools. Executing this on a Read Replica isolates the primary write database completely.

4. Edge Cases & Reliability

- **Idempotency:** Design the settlement_records table with a unique composite key: UNIQUE KEY uidx_merchant_period (merchant_id, settlement_date). Use an upsert strategy (INSERT INTO ... ON DUPLICATE KEY UPDATE or delete-then-insert) so that running the job multiple times does not result in duplicate settlements.
- **Resumability:** Track job states in a database checkpoint table (e.g., batch_job_execution). Store the last_processed_merchant_id after each successful batch commit. If the job fails and is restarted, it queries this checkpoint table and resumes from where it crashed.

PART III: AI Coding Proficiency

Question 9: AI Coding

Part 1: Code Review

1. **SQL Injection Vulnerability:**

- *Problem:* The code concatenates strings directly to form the insert statement:
`"INSERT INTO users (name, email, age) VALUES ('" + name +`
`...`
 - *Incident Scenario:* If a CSV file contains a name payload like `Jack ' , 'jack@test.com' , 25)`; `DROP TABLE users; --`, the database will execute the injection payload, dropping the table or exposing private data.
2. **Resource Leak (Database Connections and Readers):**
- *Problem:* Database connections and file readers are closed at the end of the `try` block. If an exception occurs during loop execution, the closing calls are bypassed.
 - *Incident Scenario:* A parse exception on row 10 leaves the JDBC connection open. Over time, successive file imports exhaust the database connection pool, leading to connection timeouts and application downtime.
3. **Severe Performance Bottleneck (No Batching):**
- *Problem:* Every single valid row triggers a separate network round-trip to execute an individual insert (`stmt.executeUpdate()`).
 - *Incident Scenario:* Importing a CSV with \$50,000\$ rows will make \$50,000\$ sequential database calls, taking several minutes or hours and saturating database active threads.
4. **Flawed Validation Logic:**
- *Problem:* The code validates email strings using `!email.contains("@")`, which matches invalid strings like `@` or `email@`.
 - *Incident Scenario:* Malformed email addresses will bypass validation, polluting the database with invalid dirty data that crashes downstream notification services.

Part 2: Prompt Improvement

Improved Prompt

Write a production-ready Java class using Spring Boot and JDBC to import user data from a CSV file. The class must expose a method `public int importUsersFromCsv(String filePath)`.

Please ensure the implementation satisfies these strict production standards:

1. **Security:** Use `PreparedStatement` with bind variables to guarantee complete protection against SQL Injection.
2. **Resource Management:** Use Java's `try-with-resources` statement to guarantee safe closure of DB connections, statements, and file streams under all success and exception execution paths.
3. **Performance:** Implement batch insertion using JDBC batch updates with a configurable batch size (e.g., \$1000\$ records per batch) to optimize database network I/O.

4. **Validation:** Implement a secure, regex-based email validation utility, and enforce that user age must be ≥ 18 .
5. **Robust Error Handling:** Wrap parsing and DB errors on individual records so that a single corrupted row does not abort the entire batch import. Log skipped records and failures cleanly.